

MOCK GAME PROVIDER

Integration API Guide

Basic seamless-wallet integration for iframe and end-to-end testing

Version: 1.0

Repository: Boki0/mock-game-provider

Scope: Launch, Authenticate, Bet, Result, Session lookup and Close

This document explains what an operator must implement to integrate with the mock provider. It focuses on external contracts and request flow. Internal provider implementation details are intentionally omitted.

Table of Contents

1. Integration overview
2. Current readiness and limitations
3. Environment and startup
4. Provider catalog
5. Game launch
6. Session authentication
7. Bet callback
8. Result callback
9. Session status and close
10. Complete flow example
11. Error handling
12. Integration checklist

1. Integration Overview

The mock provider models a simplified seamless-wallet flow. The operator owns the player account, currency and wallet balance. The provider owns the game session, round identifiers, mock game outcome and game frontend.

Main flow

Operator creates launch session -> game opens -> provider authenticates player -> provider sends Bet -> operator debits wallet -> provider sends Result -> operator credits winnings.

System	Responsibility
Operator	Creates secure token, owns player identity, currency and wallet balance, processes Authenticate, Bet and Result callbacks.
Mock Provider	Creates session and launch URL, hosts game frontend, generates round/reference values and calls operator wallet endpoint.
Browser Game	Sends selected bet amount to provider backend and displays balances returned by the operator through the provider.

The browser never receives the private operator token and does not calculate the wallet balance locally.

2. Current Readiness and Limitations

Assessment: the project is sufficient for a basic local or test-environment integration of the primary happy path.

Capability	Status	Notes
Provider/game catalog	Available	Providers and games can be listed.
Game launch	Available	Creates an in-memory provider session and returns a launch URL.
Authenticate	Available	Provider calls the operator and validates player/currency/balance.
Bet	Available	Operator debit callback with basic response validation.
Result	Available	Operator credit callback using the same roundId and a different reference.
Session lookup/close	Available	Public session does not expose token.
Refund	Not implemented	Crucial next step for recovering a debited bet when a round cannot complete.
EndRound	Not implemented	Useful for explicit round finalization; optional for a minimal demo.
Persistence	Not implemented	Sessions, rounds and idempotency data are lost on restart.
Request signing	Not implemented	JSON only; no MD5/HMAC/API key verification.
Production retry/reconciliation	Not implemented	No durable retries or reconciliation mechanism.

Recommendation: Add Refund before calling the integration complete beyond a demo. EndRound can follow as the next small step. Persistence and request signing are required before any production-style use.

3. Environment and Startup

Provider configuration

```
OPERATOR_BASE_URL=http://localhost:8080
PROVIDER_BASE_URL=http://localhost:8090
PORT=8090
SESSION_TTL_MS=900000
```

Only OPERATOR_BASE_URL is required for wallet callbacks. PROVIDER_BASE_URL is used when the provider builds the launch URL.

Start

```
npm install
OPERATOR_BASE_URL=http://localhost:8080 npm start
```

Health check

```
GET /health

Response:
{"status": "UP"}
```

4. Provider Catalog

List providers

```
GET /api/providers
```

List games

```
GET /api/games
GET /api/games?providerCode=NOVA_REELS
```

A launch request must use a valid providerCode and a gameCode that belongs to that provider. The current sample game frontend is launched under /games/lucky-seven/.

5. Game Launch

Request

```
POST /api/launch
Content-Type: application/json

{
  "providerCode": "NOVA_REELS",
  "gameCode": "NOVA_SEVEN",
  "playerId": "player-123",
  "currency": "EUR",
  "token": "operator-generated-token",
  "mode": "REAL"
}
```

Response

```
{
  "sessionId": "provider-session-id",
  "launchUrl": "http://localhost:8090/games/lucky-seven/?sessionId=provider-session-id"
}
```

The token is stored only in the provider session. It is not included in the public session response or returned to the browser.

6. Session Authentication

When the game opens, the frontend verifies the session and asks the provider backend to authenticate it.

Browser to provider

```
POST /api/sessions/{sessionId}/authenticate

Empty body
```

Provider to operator

```
POST {OPERATOR_BASE_URL}/api/provider-wallet/authenticate
Content-Type: application/json

{
  "token": "operator-generated-token",
  "providerCode": "NOVA_REELS",
  "gameCode": "NOVA_SEVEN",
  "sessionId": "provider-session-id"
}
```

Expected operator response

```
{
  "userId": "player-123",
  "currency": "EUR",
  "cash": 1000.00,
  "bonus": 0.00,
  "error": 0,
  "description": "Success"
}
```

The provider accepts playerId instead of userId as well. The returned player and currency must match the launch session.

7. Bet Callback

The browser sends only the selected amount. The provider adds trusted session data and generated transaction identifiers.

Browser to provider

```
POST /api/sessions/{sessionId}/bet
Content-Type: application/json

{
  "amount": 20.00
}
```

Provider to operator

```
POST {OPERATOR_BASE_URL}/api/provider-wallet/bet
Content-Type: application/json

{
  "userId": "player-123",
  "gameId": "NOVA_SEVEN",
  "roundId": "provider-round-id",
  "amount": 20.00,
  "reference": "provider-bet-reference",
  "providerId": "NOVA_REELS",
  "timestamp": 1780000000000,
  "roundDetails": "spin",
  "platform": "WEB",
  "language": "en",
}
```

```
"token": "operator-generated-token",
"ipAddress": "127.0.0.1"
}
```

Expected operator response

```
{
  "transactionId": "operator-bet-tx-id",
  "currency": "EUR",
  "cash": 980.00,
  "bonus": 0.00,
  "usedPromo": 0.00,
  "error": 0,
  "description": "Success"
}
```

On success, the provider returns the operator balance to the game and starts the spin animation. The operator must not debit the same reference more than once.

8. Result Callback

After the game calculates the gross win, it completes the existing round. Result uses the same roundId as Bet and a new reference.

Browser to provider

```
POST /api/rounds/{roundId}/result
Content-Type: application/json

{
  "amount": 500.00,
  "roundDetails": "spin"
}
```

Provider to operator

```
POST {OPERATOR_BASE_URL}/api/provider-wallet/result
Content-Type: application/json

{
  "userId": "player-123",
  "gameId": "NOVA_SEVEN",
  "roundId": "provider-round-id",
  "amount": 500.00,
  "reference": "provider-result-reference",
  "providerId": "NOVA_REELS",
  "timestamp": 1780000001000,
  "roundDetails": "spin",
  "platform": "WEB",
  "token": "operator-generated-token"
}
```

Expected operator response

```
{
  "transactionId": "operator-result-tx-id",
  "currency": "EUR",
  "cash": 1480.00,
  "bonus": 0.00,
  "error": 0,
  "description": "Success"
}
```

Meaning of amount: amount is the gross credit. A value of 0 completes a losing round. For 1000 initial cash, Bet 20 and Result 500, the final balance is 1480.

9. Session Status and Close

Get public session

```
GET /api/sessions/{sessionId}
```

The response includes session status, player, game, currency, balances and timestamps, but never includes the private token.

Close session

```
POST /api/sessions/{sessionId}/close
```

HTTP	Meaning
200	Active session was closed, or an already closed session was returned.
404	Session does not exist.

HTTP	Meaning
409	Expired session cannot be closed.

10. Complete Flow Example

Step	Direction	Action	Balance example
1	Operator -> Provider	POST /api/launch	1000
2	Game -> Provider	Authenticate session	1000
3	Provider -> Operator	Authenticate callback	1000 returned
4	Game -> Provider	Bet amount 20	unchanged in browser until response
5	Provider -> Operator	Bet callback	operator debits to 980
6	Provider -> Game	Bet accepted	980 displayed
7	Game	Spin animation and mock outcome	no local wallet change
8	Game -> Provider	Result amount 500	980
9	Provider -> Operator	Result callback	operator credits to 1480
10	Provider -> Game	Round completed	1480 displayed

Bet and Result are separate wallet transactions. They share roundId but use different reference values.

11. Error Handling

Provider HTTP	Typical meaning
400	Missing or invalid field, invalid amount or unsupported mode.
404	Provider, game, session or round was not found.
409	Session/round state does not allow the operation.
422	Operator explicitly rejected Bet or Result.
502	Operator request failed or operator response was invalid.

Operator success contract

Operator callbacks should return HTTP 2xx and JSON with error equal to 0 or "0". Bet and Result responses must contain transactionId, currency, cash and bonus. Bet also returns usedPromo.

Idempotency

The operator must process each Bet and Result reference at most once. The provider keeps basic in-memory responses. Because this state is not durable, restart-safe idempotency is not provided.

12. Integration Checklist

Check	Required
Expose POST /api/provider-wallet/authenticate	Yes
Expose POST /api/provider-wallet/bet	Yes
Expose POST /api/provider-wallet/result	Yes
Validate launch token and map it to the expected player	Yes
Return the same currency used during launch	Yes
Keep the wallet balance only on operator side	Yes
Make Bet idempotent by reference	Yes
Make Result idempotent by reference	Yes
Configure OPERATOR_BASE_URL	Yes
Implement Refund before failure-safe integration	Strongly recommended
Implement EndRound	Recommended
Add durable storage and request signing	Required for production-style use

Final assessment: Ready for basic happy-path integration and local end-to-end testing. The most important missing wallet operation is Refund. The project should not be described as production-ready until durable idempotency, persistence, authentication/signing, retries and reconciliation are added.

Source basis: public repository state reviewed on 23 July 2026 and the uploaded Pragmatic Play Integration API specification used only as a structural reference.